

● YOUR CAR YOUR WAY

OPTION B · SCÉNARIO FICTIF · SOUTENANCE

Application web centralisée

Analyse · Architecture · Preuve de concept

Présentation des livrables — 15 minutes

Redelberger Sacha

Architecte logiciel · Mai 2026

Your Car Your Way aujourd'hui

- › Entreprise de location de voitures — 20+ ans
- › Présence Europe + Amérique du Nord
- › **4 applications distinctes** développées indépendamment
- › Aucune stratégie d'unification technique
- › 100 % monolithes — aucun microservice

Problème 1 — Complexité

Java EE, PHP Laravel, Node.js, Spring Boot : 4 stacks différents à maintenir.

Problème 2 — Incohérence

Un client européen au Canada ne voit pas ses réservations européennes.

Problème 3 — Risques légaux

SHA-1 encore utilisé en France. TLS 1.0 actif. Aucun audit WCAG documenté.

5 personas identifiés

Profil	Besoin clé
 Marc — Client standard	Compte unique cross-marchés
 Sophie — Malvoyante PSH	<code>alt</code> , <code>aria-live</code> , focus clavier
 Julien — Handicap moteur PSH	Navigation au clavier, switch
 Amina — Agent support	Tchat simultané, fiche client
 Thomas — Administrateur	Back-office centralisé, logs

Considérations transverses

 ACCESSIBILITÉ — WCAG 2.1 AA

Obligation légale RGAA · Principes POUR · `aria-live` sur le tchat

 RGPD — DROITS UTILISATEURS

Accès · Rectification · Effacement (anonymisation `SET_NULL`) · Portabilité

 INTERNATIONALISATION (I18N)

FR · EN · ES · Fichiers `.po` · Formats dates & monnaies locaux

97,2%

Dispo FR/DE/ES/IT
(cible : 99,5%)

2h45

MTTR OVH
(cible : < 1h)

SHA-1

Hachage MDP FR
cryptographiquement cassé

41%

Packages vulnérables
côté France

98,9%

Dispo USA
Référence cible

Application	Stack	Disponibilité	Sécurité MDP	Déploiement
FR + DE/ES/IT	Java EE / JSP	97,2 %	SHA-1 ⚠	Manuel (82%)
Royaume-Uni	PHP Laravel	98,6 %	bcrypt	AWS (91%)
Canada	Node.js / React	98,1 %	argon2id	AWS (91%)
États-Unis	Spring Boot / Angular	98,9 %	bcrypt	Azure containers

Pourquoi le monolithe modulaire ?

Option	Décision	Raison
Microservices	✗ Écarté	Complexité trop élevée
Serverless	✗ Écarté	WebSocket difficile
Monolithe modulaire	✓ Retenu	Simple, cohérent, scalable

Stack technique

 Django 4.2

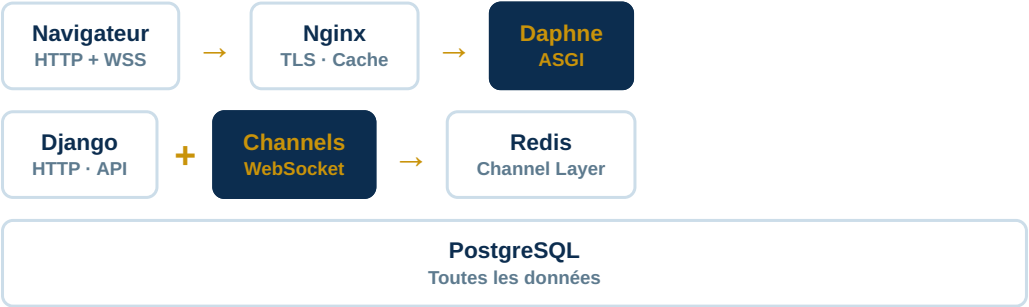
 Channels 4

 Redis

 PostgreSQL

 Daphne

Flux de l'architecture



● DJANGO 4.2 LTS

- › **ORM intégré** → pas de SQL brut
- › **Admin auto-généré** → semaines économisées
- › **Auth + i18n** natifs
- › **LTS** → sécurité jusqu'en 2026
- › **FastAPI** : pas d'admin. **Node** : pas d'ORM.

🐘 POSTGRESQL 15

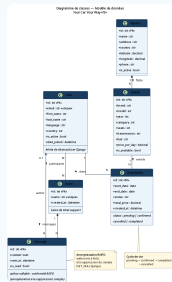
- › **Schéma strict** → données cohérentes
- › **Transactions ACID** → réservations sûres
- › **ORM Django natif premium**
- › **MongoDB** → cause des problèmes dans l'audit
- › **PostGIS** disponible pour géolocalisation

⚡ CHANNELS + REDIS

- › **WebSocket** → tchat temps réel
- › Extension **officielle** Django
- › **Redis** = diffusion entre workers
- › Pas de second service Node.js
- › **Pusher** = données chez un tiers (RGPD ❌)

Cohérence : pourquoi Daphne et pas Gunicorn ?

Gunicorn est WSGI (synchrone) — il ne supporte pas les WebSockets. Daphne est ASGI (asynchrone) — il gère HTTP et WebSocket simultanément. Obligatoire dès qu'on utilise Django Channels.



3 décisions de conception notables

Room.participants — Many-to-Many

Un salon peut avoir plusieurs agents. Un utilisateur peut avoir plusieurs rooms. La relation M2M représente cette réalité.

Message.author — Nullable (RGPD)

`SET_NULL` Django : à la suppression du compte, l'auteur devient NULL. Le message est conservé (utile pour les litiges) mais anonymisé.

Agency.is_active — Désactivation logique

Fermer une agence temporairement sans supprimer l'historique des réservations passées.

Ce que la PoC démontre

Test	Technologie validée
Connexion WebSocket instantanée	Daphne + Django Channels + ASGI
Messages en temps réel multi-onglets	Redis Channel Layer
Historique au rechargement	PostgreSQL + ORM Django

Tests unitaires — 40 / 40

17

Tests modèles
(RGPD validé)

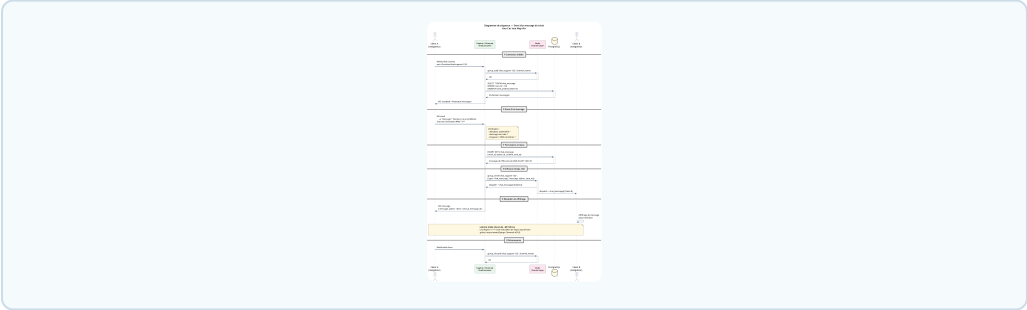
12

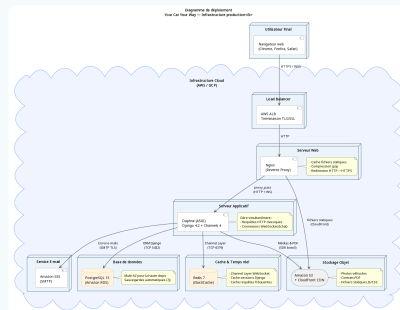
Tests serializers
(API JSON)

11

Tests vues
(HTTP 200/404)

Flux WebSocket





Infrastructure production

- **Nginx** — TLS, cache, load balancing
- **Daphne** — HTTP + WebSocket (ASGI)
- **PostgreSQL RDS** — Base multi-AZ
- **Redis ElastiCache** — Channel Layer

- ▶ **S3 + CloudFront** — Médias & Statiques
- ▶ **Docker** — Conteneurisation
- ▶ Dev = Recette = Production
- ▶ **Amazon SES** — Envoi d'e-mails
- ▶ Reproductibilité garantie

Les 3 livrables produits

Livrable	Contenu
Cahier des charges	5 personas · 12 US + CA · MoSCoW · WCAG · RGPD
Proposition d'architecture	Audit · Monolithe Django · 4 diagrammes UML
PoC tchat	WebSocket + Redis + PostgreSQL · 40 tests

Réponse aux enjeux

- ✓ **Évolutivité**

Monolithe modulaire → microservices si le trafic le justifie. API REST déjà prête pour une app mobile.
- ✓ **Maintenabilité**

Un seul langage (Python), une seule base (PostgreSQL), une seule équipe. Django LTS 2026.
- ✓ **Conformité**

WCAG 2.1 AA (aria-live, focus clavier) · RGPD (SET_NULL, export JSON) · bcrypt pour tous les MDP.